DAA ASSIGNMENT 2026 — ONE-PAGE ANALYSIS SHEET

Batch 12 | Disagreement Between Two Recommenders (OTT / Streaming) | Course: 23I501 — Design and Analysis of Algorithms

1. Problem and model

Engine A and Engine B rank the same **n unique titles**. Convert Engine B's ranking into Engine A positions. An inversion is a pair (i, j) where i < j and B[i] > B[j]. The output is the total number of inversions (a counting problem). Example input: **[3, 1, 6, 2, 8, 4, 7, 5]**; expected count: **9**.

2. Data structures and operation costs

Structure / method	Important operations	Total time	Extra space	Best use
Array + pair scan	Read each pair once; compare B[i] and B[j]. No updates.	$O(n^2)$	O(1)	Small inputs / baseline
Array + merge sort	Split recursively; merge sorted halves; add remaining-left count for each cross-inversion.	O(n log n)	O(n)	Full batch ranking
Fenwick tree (BIT)	Point update: O(log n); prefix-sum query: O(log n); coordinate compression: O(n log n).	O(n log n)	O(n)	Incremental arrivals / prefix queries

3. Selected technique and correctness

Divide and Conquer — Merge Sort. Divide the array into two halves, recursively count inversions in each half, then count cross-inversions while merging. If right[j] < left[i], the left half is sorted, so all remaining left values are greater than right[j]; add (len(left) − i). Every inversion is exactly one of: left-half, right-half, or cross-half. The recursive calls count the first two categories and the merge counts the third, so the sum is correct. Base case: size 0 or 1 has zero inversions.

4. Hand trace and complexity

Trace subtotals: [3] vs [1] = 1; [6] vs [2] = 1; merge [1,3] with [2,6] adds 1 (left subtotal 3). Right side: [8] vs [4] = 1; [7] vs [5] = 1; merge [4,8] with [5,7] adds 2 (right subtotal 4). Final merge adds 2. **Total = 3 + 4 + 2 = 9.** Recurrence: T(n) = 2T(n/2) + O(n), hence O(n log n) time and O(n) auxiliary space. Best, average and worst-case time remain O(n log n); reverse order has the maximum n(n–1)/2 inversions (28 when n=8).

5. Empirical results (5 trials; median seconds)

n	Merge sort	Brute force	Inversions
100	0.000070148	0.000130895	2,151
500	0.000479754	0.003889095	65,826
1,000	0.001086131	0.016840278	246,619
2,000	0.002326943	0.067572508	972,436
5,000	0.006626749	0.425294680	6,132,879

Measured on deterministic random permutations; both implementations were checked for matching answers before timing. At n=5,000, merge sort took about 0.0066 s versus 0.4253 s for pair scanning. Timings depend on hardware and Python version.

6. Other techniques and changed requirement

Greedy: a local choice does not count all inverted pairs. **Dynamic Programming:** recursive halves do not overlap, so memoization adds no benefit. **Backtracking:** enumerating possible rankings is unnecessary and grows factorially; direct pair scanning is $O(n^2)$. **Branch and Bound:** every inversion must be counted, so pruning a search tree cannot avoid the required pair accounting. **Weighted top-10 variant:** each inversion involving a top-10 title gets weight 5; other inversions get weight 1. Current implementation assumes a title is top-10 if it is in Engine A's OR Engine B's top 10. Confirm this interpretation with the instructor. For the given n=8 example, all titles qualify: weighted score = 9 × 5 = **45**.

7. Reference and submission note

Foundational reference: Donald E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998. This supports the general sorting/permutation background; it does **not** establish that a named OTT platform uses this exact recommender-disagreement pipeline. Add a verified real-world system or published case study if the assignment requires an industry implementation. Benchmark source: Batch_12/results/timing.csv. Algorithm: code/main_solution.py; baseline: code/rejected_alternative.py.